

Terraform final task

Tasks:

1. Create VPC
2. Create Internet gateway
3. Create Custom Route Table
4. Create Subnet
5. Associate subnet with Route Table
6. Create Security Group to allow port 22,80,443
7. Create a network interface with an ip in the subnet that was created in step 4
8. Assign an elastic IP to the network interface created in step 7
9. Create Ubuntu server and install/enable apache2

Task 1: create vpc

Steps

Step 1: create a folder and inside folder create a file name Main.tf

Step 2: add the below code to create a vpc

Step 3 : after adding the code in terminal add terraform init (to initialize the .terraform)

```
main.tf > ...
1  provider "aws" {
2      region = "us-east-1"
3  }
4
5  # 1) Create VPC
6  resource "aws_vpc" "myvpc" {
7      cidr_block = "10.0.0.0/16"
8
9      tags = {
10         Name = "myvpc"
11     }
12 }
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
+ owner_id                = (known after apply)
+ region                  = "us-east-1"
+ tags                    = {
  + "Name" = "myvpc"
}
+ tags_all                 = {
  + "Name" = "myvpc"
}
}
```

Plan: 9 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ public_ip = (known after apply)
```

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.myvpc: Creating...

aws_vpc.myvpc: Creation complete after 5s [id=vpc-0c4548442b04f2c67]

Step 4 : terraform plan --> after plan

Step 5 : terraform apply -auto-approve

∴ now after the result is resource added (means we successfully created vpc)

Step 6: now go and check you aws us-east-1 region vpc

Your VPCs

VPCs

VPC encryption controls

Your VPCs (2)

Info

Last updated less than a minute ago

Actions

Create V

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR
☐	myvpc	vpc-0c4548442b04f2c67	Available	-	-	Off	10.0.0.0/16
☐	default	vpc-07968786753c2ea72	Available	-	-	Off	172.31.0.0/16

```
tags = {
  Name = "myvpc"
}
```

—> we can see we have given tag name of vpc is same “myvpc”

Task 2 : Create Internet gateway

—-> Add below code in your main.tf

```
13
14 # 2) Create Internet Gateway
15 resource "aws_internet_gateway" "igw" {
16   vpc_id = aws_vpc.myvpc.id
17
18   tags = {
19     Name = "my-igw"
20   }
21 }
22
```

—-> terraform apply

And check your aws internet gateways

Internet gateways (2) Info						Actions	Create internet
Find internet gateways by attribute or tag							< 1
☐	Name	Internet gateway ID	State	VPC ID	Owner		
☐	my-igw	igw-0404e11a0e1c47cf1	✔ Attached	vpc-0c4548442b04f2c67 myvpc	207662791773		
☐	default-vpc	igw-05d93d6a24c45086c	✔ Attached	vpc-07968786753c2ea72 default	207662791773		

```
tags = {
  Name = "my-igw"
}
```

Task 3: Create Custom Route Table

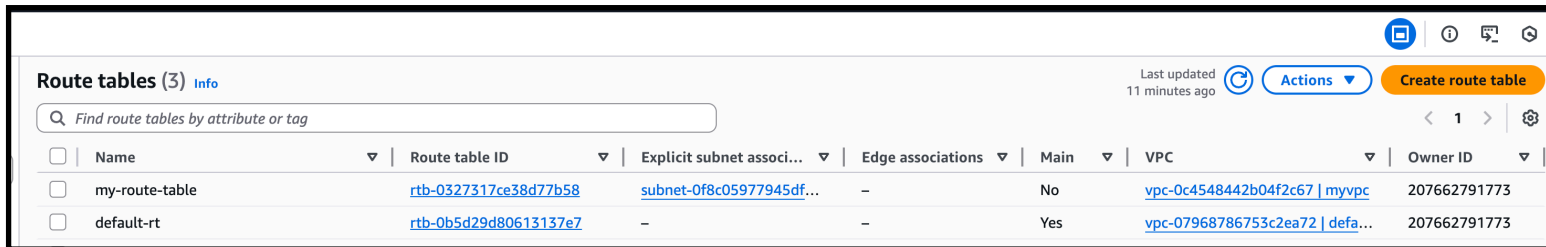
```

23  # 3) Create Custom Route Table
24  resource "aws_route_table" "rt" {
25      vpc_id = aws_vpc.myvpc.id
26
27      route {
28          cidr_block = "0.0.0.0/0"
29          gateway_id = aws_internet_gateway.igw.id
30      }
31
32      tags = {
33          Name = "my-route-table"
34      }
35  }
36

```

- > write the above code and
- > terraform apply

--> now check your aws route table



☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
☐	my-route-table	rtb-0327317ce38d77b58	subnet-0f8c05977945df...	-	No	vpc-0c4548442b04f2c67 myvpc	207662791773
☐	default-rt	rtb-0b5d29d80613137e7	-	-	Yes	vpc-07968786753c2ea72 defa...	207662791773

```
tags = {  
  Name = "my-route-table"  
}
```

Task 4: create subnet

```
# 4) Create Subnet  
resource "aws_subnet" "subnet1" {  
  vpc_id           = aws_vpc.myvpc.id  
  cidr_block       = "10.0.1.0/24"  
  availability_zone = "us-east-1a"  
  map_public_ip_on_launch = true  
  
  tags = {  
    Name = "my-subnet"  
  }  
}
```

Subnets (1/1) Info

Last updated 6 minutes ago

Actions

Create subnet

Find subnets by attribute or tag

VPC : vpc-0c4548442b04f2c67

Clear filters

☒	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR
☒	my-subnet	subnet-0f8c05977945dfbc5	Available	vpc-0c4548442b04f2c67 myvpc	Off	10.0.1.0/24	-

Task 5: Associate subnet with Route Table

```
49 # 5) Associate Subnet with Route Table
50 resource "aws_route_table_association" "rta" {
51     subnet_id      = aws_subnet.subnet1.id
52     route_table_id = aws_route_table.rt.id
53 }
54
```

Route tables (1/3) Info

Last updated 18 minutes ago

Actions

Create route table

Find route tables by attribute or tag

☒	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
☒	my-route-table	rtb-0327317ce38d77b58	subnet-0f8c05977945df...	-	No	vpc-0c4548442b04f2c67 myvpc	207662791773
☐	default-rt	rtb-0b5d29d80613137e7	-	-	Yes	vpc-07968786753c2ea72 defa...	207662791773
☐	-	rtb-0350d6a88a53aff01	-	-	Yes	vpc-0c4548442b04f2c67 myvpc	207662791773

rtb-0327317ce38d77b58 / my-route-table

Details | Routes | Subnet associations | Edge associations | Route propagation | Tags

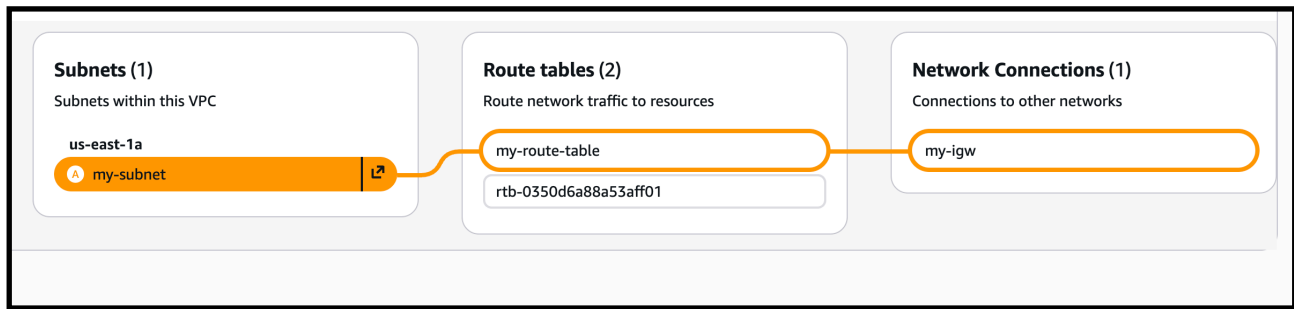
Explicit subnet associations (1)

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
my-subnet	subnet-0f8c05977945dfbc5	10.0.1.0/24	-

Subnets without explicit associations (0)

Edit subnet associations



Task 6: Create Security Group to allow port 22 , 80 , 443

```
5 # 6) Create Security Group (Allow 22,80,443)
6 resource "aws_security_group" "web_sg" {
7   name           = "web-sg"
8   description    = "Allow SSH HTTP HTTPS"
9   vpc_id         = aws_vpc.myvpc.id
10
11   ingress {
12     description = "SSH"
13     from_port   = 22
14     to_port     = 22
15     protocol    = "tcp"
16     cidr_blocks = ["0.0.0.0/0"]
17   }
18
19   ingress {
20     description = "HTTP"
21     from_port   = 80
22     to_port     = 80
23     protocol    = "tcp"
24     cidr_blocks = ["0.0.0.0/0"]
25   }
26
27   ingress {
28     description = "HTTPS"
29     from_port   = 443
30     to_port     = 443
31     protocol    = "tcp"
32     cidr_blocks = ["0.0.0.0/0"]
33   }
34
35   egress {
36     from_port   = 0
37     to_port     = 0
38     protocol    = "-1"
39     cidr_blocks = ["0.0.0.0/0"]
40   }
41
42   tags = {
43     Name = "web-sg"
44   }
45 }
```

Security Groups (1/2) Info

Find security groups by attribute or tag

VPC ID = vpc-0c4548442b04f2c67

Clear filters

Actions

Export security groups to CSV

Create security group

	Name	Security group ID	Security group name	VPC ID	Description
☐	-	sg-0a7b4b33bacc1fac5	default	vpc-0c4548442b04f2c67	default VPC security group
☒	web-sg	sg-0d8cede3de236dc70	web-sg	vpc-0c4548442b04f2c67	Allow SSH HTTP HTTPS

Security Groups > sg-0d8cede3de236dc70 - web-sg

sg-0d8cede3de236dc70 - web-sg

Details

Security group name

web-sg

Owner

207662791773

Security group ID

sg-0d8cede3de236dc70

Inbound rules count

3 Permission entries

Description

Allow SSH HTTP HTTPS

Outbound rules count

1 Permission entry

VPC ID

vpc-0c4548442b04f2c67

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (3)

Manage tags

Edit inbound rules

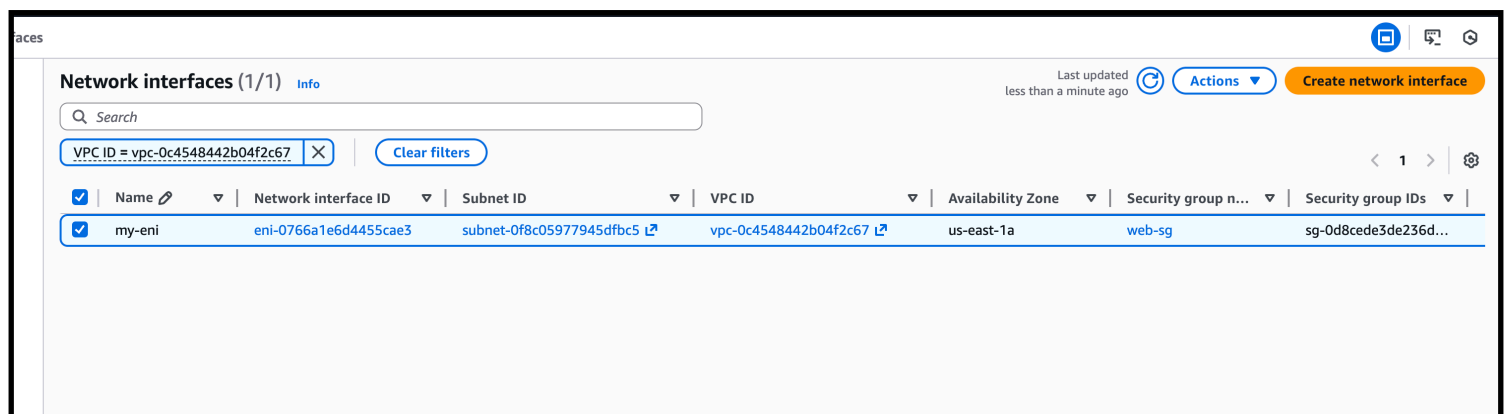
	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sgr-0ee1f74aa779132d7	IPv4	HTTPS	TCP	443	0.0.0.0/0
☐	-	sgr-029c60eeb1d776427	IPv4	SSH	TCP	22	0.0.0.0/0
☐	-	sgr-0c2e86add345900f6	IPv4	HTTP	TCP	80	0.0.0.0/0

Task 7: Create a network interface with an ip in the subnet that was created in step 4

```
# 7) Create Network Interface with IP in subnet
resource "aws_network_interface" "eni" {
  subnet_id      = aws_subnet.subnet1.id
  private_ips    = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]

  tags = {
    Name = "my-eni"
  }
}
```

—> in aws check network interfaces



The screenshot shows the AWS Management Console interface for 'Network interfaces (1/1)'. The page includes a search bar, a filter for 'VPC ID = vpc-0c4548442b04f2c67', and a 'Clear filters' button. The main table lists the network interface 'my-eni' with the following details:

Name	Network interface ID	Subnet ID	VPC ID	Availability Zone	Security group n...	Security group IDs
my-eni	eni-0766a1e6d4455cae3	subnet-0f8c05977945dfbc5	vpc-0c4548442b04f2c67	us-east-1a	web-sg	sg-0d8cede3de236d...

Task 8: Assign an elastic IP to the network interface created in step 7

```
# 8) Assign Elastic IP to the network interface
resource "aws_eip" "eip" {
  domain                = "vpc"
  network_interface     = aws_network_interface.eni.id
  associate_with_private_ip = "10.0.1.50"

  depends_on = [
    aws_internet_gateway.igw,
    aws_instance.ubuntu
  ]

  tags = {
    Name = "my-eip"
  }
}
```

Elastic IP addresses (1/1) [Info](#)

[Actions](#) [Allocate Elastic IP address](#)

☒	Name	Allocated IPv4 address	Type	Allocation ID	Reverse DNS record	Associated instance ID
☒	my-eip	54.221.231.61	Public IP	eipalloc-086fae0d4b6a3efa0	-	i-09f4ae87ad5f53cd4 ↗

54.221.231.61

[View IP address usage and recommendations to release unused IPs with \[Public IP insights\]\(#\).](#)

[Summary](#) [Tags](#)

Summary

Allocated IPv4 address
[54.221.231.61](#)

Association ID
[eipassoc-07f12bad45a762fc7](#)

Network interface ID
[eni-0766a1e6d4455cae3](#)

Address pool
[Amazon](#)

Type
[Public IP](#)

Scope
[VPC](#)

Network interface owner account ID
[207662791773](#)

Network border group
[us-east-1](#)

Allocation ID
[eipalloc-086fae0d4b6a3efa0](#)

Associated instance ID
[i-09f4ae87ad5f53cd4](#)

Public DNS
-

Service managed
-

Reverse DNS record
-

Private IP address
[10.0.1.50](#)

NAT Gateway ID
-

Task 9: Create Ubuntu server and install/enable apache2

```
# 9) Create Ubuntu EC2 and install/enable Apache2
resource "aws_instance" "ubuntu" {
  ami          = "ami-0b6c6ebed2801a5cb"
  instance_type = "t3.small"
  key_name     = "jenkins"

  primary_network_interface {
    network_interface_id = aws_network_interface.eni.id
  }

  user_data = <<-EOF
  #!/bin/bash
  sudo apt update -y
  sudo apt install apache2 -y
  sudo systemctl start apache2
  sudo systemctl enable apache2
  echo "<h1>Apache2 Installed Successfully</h1>" | sudo tee /var/www/html/index.html
  EOF

  tags = {
    Name = "Ubuntu-Web-Server"
  }
}
```

☐	jenkins-slave	i-0bf14b0eb013d170a	Stopped		t3.micro	-	View alarms	us-east-1a	-	-
☒	Ubuntu-Web-...	i-09f4ae87ad5f53cd4	Running		t3.small	3/3 checks passed	View alarms	us-east-1a	-	54.221.231.
☐	sonarqube	i-01cb0fac0199e8107	Stopped		m7i-flex.large	-	View alarms	us-east-1b	-	-
☐	sonarqube2	i-08b064238eac3cac0	Stopped		m7i-flex.large	-	View alarms	us-east-1b	-	-

```

Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Mon Jan 26 07:04:28 UTC 2026

System load:  0.62           Temperature:   -273.1 C
Usage of /:   30.7% of 6.71GB Processes:      128
Memory usage: 19%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 10.0.1.50

Expanded Security Maintenance for Applications is not enabled.

57 updates can be applied immediately.
29 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-1-50:~$ sudo -i

```

- > check apach2 running or not
- > systemctl status apache2

```

root@ip-10-0-1-50:~# systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-01-26 07:04:23 UTC; 50min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1934 (apache2)
    Tasks: 55 (limit: 2204)
   Memory: 5.7M (peak: 6.1M)
      CPU: 195ms
   CGroup: /system.slice/apache2.service
           └─1934 /usr/sbin/apache2 -k start
             └─1937 /usr/sbin/apache2 -k start
               └─1938 /usr/sbin/apache2 -k start

Jan 26 07:04:23 ip-10-0-1-50 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 26 07:04:23 ip-10-0-1-50 apachectl[1933]: AH00558: apache2: Could not reliably determine the server's fully qualifie
Jan 26 07:04:23 ip-10-0-1-50 systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-16/16 (END)

```

